

MILITARY INSTITUTE OF SCIENCE AND TECHNOLOGY

Mirpur Cantonment

Department of Computer Science and Engineering

TERM PAPER

Course: Operating Systems (CSE-307)

**Learning-Augmented Disk Scheduling
Under Changing Workload Patterns**

Prepared by:

Md. Arif Sadik Molla

ID: 202414046

Level 3 [CSE-24]

Submitted to:

Lec. Khaled Hasan

Department of Computer Science and Engineering

Military Institute of Science and Technology

October 4, 2026

Abstract

Disk scheduling performance depends strongly on request order and locality. This project compares FCFS, SSTF, SCAN, and C-SCAN using a reproducible simulation and adds a lightweight Decision Tree that selects a scheduler for each workload window. The simulated disk contains cylinders 0 to 199, the initial head position is 50, and each window contains 40 requests. Training labels are generated by running all four schedulers on the same workload and selecting the one with the lowest total head movement. The Decision Tree achieved 84.38% test accuracy. SSTF performed best for most uniform, localized, and bursty workloads, while FCFS was competitive for directional workloads. The learned selector usually selected an effective policy but did not always match the oracle.

1. Introduction

Disk scheduling determines the order in which pending disk requests are serviced. In the traditional mechanical disk model, this ordering affects disk-head movement, so different scheduling policies can produce very different costs for the same request set [1, 2].

FCFS follows arrival order, SSTF favors nearby requests, and SCAN-family algorithms use directional sweeps. Their performance therefore depends on the structure of the workload. This project investigates whether a simple learned selector can choose among these classical algorithms according to recent request characteristics.

The work is simulation-based. It does not modify an operating-system kernel or measure physical HDD latency. Total head movement in cylinders is used as a seek-distance proxy.

GitHub Repository:

<https://github.com/Arif-Sadik/OS-Learning-Augmented-Disk-Scheduling>

2. Methodology

2.1 Classical Scheduling

FCFS services requests in arrival order. SSTF repeatedly chooses the request closest to the current head, with equal-distance ties resolved by choosing the lower cylinder.

SCAN is implemented as true SCAN, not LOOK. Starting upward, the head services higher requests, moves to cylinder 199, then reverses and services lower requests.

C-SCAN is implemented as true C-SCAN. The head moves upward to cylinder 199, wraps to cylinder 0, and then continues upward. The wrap-around distance is included in total movement.

2.2 Synthetic Workloads

All experiments use cylinders 0–199, initial head position 50, and 40 requests per window. Four reproducible workload types are generated:

- **Uniform:** requests spread across the disk.

- **Localized:** requests concentrated in one region.
- **Directional:** mostly increasing requests with small variation.
- **Bursty:** locality followed by movement to another region.

Localized and bursty behavior represents the general idea that program access patterns can exhibit locality that changes over time [3].

2.3 Learning-Augmented Selection

For every generated window, workload features are extracted and all four schedulers are executed on the same request list. The scheduler with minimum total head movement becomes the class label.

The features include mean position, standard deviation, request range, minimum and maximum positions, mean and median consecutive movement, increasing/decreasing transition ratios, direction-change ratio, and distance between the starting head and workload mean.

The selector uses scikit-learn’s `DecisionTreeClassifier` [4], with maximum depth 4, minimum samples per leaf 8, and random state 307. The dataset contains 640 windows, split into 480 training and 160 testing windows.

2.4 Experimental Setup

The main evaluation uses 200 new windows, 50 for each workload type. For every window, FCFS, SSTF, SCAN, C-SCAN, the learned selector, and an oracle-best result are recorded.

The workload-shift experiment contains 24 windows: 12 directional followed by 12 uniform. The selector makes a new decision for each window. All 21 automated tests passed before the final experiments were run.

3. Results

Table 1: Mean total head movement across 50 evaluation windows per workload type.

Type	FCFS	SSTF	SCAN	C-SCAN	Learned	Oracle
Uniform	2680.20	303.26	344.18	393.62	303.26	302.18
Localized	524.24	93.32	205.58	232.98	93.32	93.32
Directional	192.22	225.28	327.48	395.16	211.16	187.98
Bursty	600.80	206.86	327.54	389.74	206.86	206.80

Table 1 shows that scheduler performance depends strongly on request structure. Uniform workloads produced very high FCFS movement because requests frequently jumped across the disk. SSTF reduced the mean from 2680.20 to 303.26 cylinders.

Localized workloads strongly favored SSTF, which matched the oracle at 93.32

cylinders. Directional workloads behaved differently. FCFS averaged 192.22 cylinders and was close to the oracle value of 187.98 because arrival order already followed a useful direction.

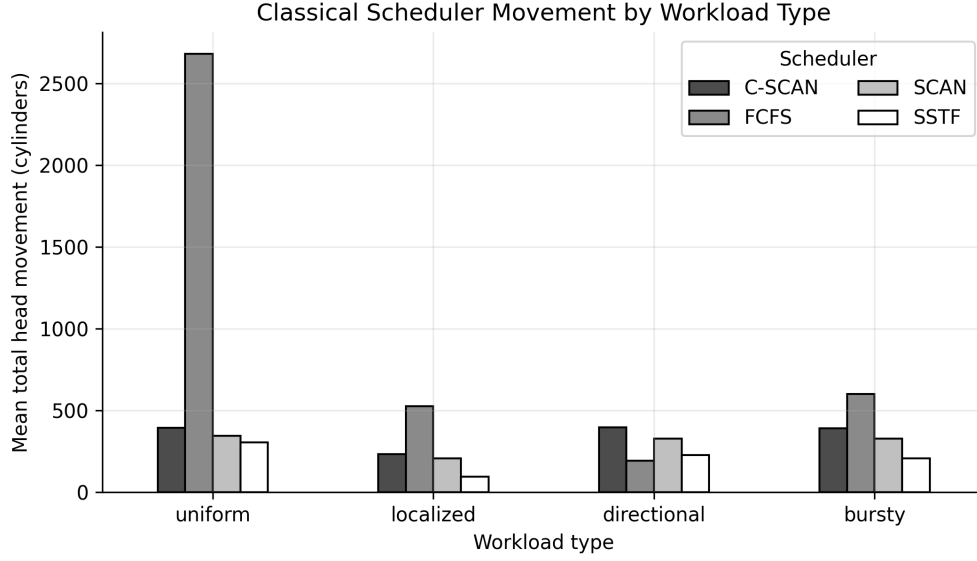


Figure 1: Classical scheduler performance by workload type.

SCAN and C-SCAN were less competitive under the selected metric because true endpoint movement was included. C-SCAN also included the full wrap from cylinder 199 to cylinder 0.

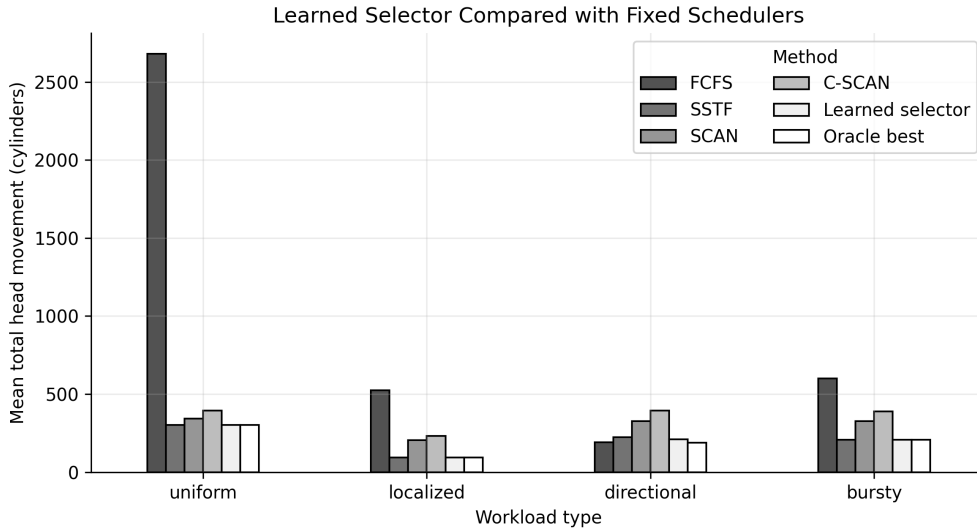


Figure 2: Learned selector compared with fixed policies and the oracle.

The Decision Tree reached 84.38% accuracy. Across the full 640-window dataset, the class distribution was FCFS 93, SSTF 535, SCAN 12, and C-SCAN 0. The zero C-SCAN labels resulted from the selected workloads and total-movement metric, not from an implementation failure.

The learned selector matched the oracle average for localized workloads. For uniform workloads, its mean movement was 303.26 compared with 302.18 for the oracle. For bursty workloads, the values were 206.86 and 206.80 respectively.

Directional workloads were more difficult, with 211.16 cylinders for the selector compared with the oracle’s 187.98.

Table 2: Workload-shift results.

Phase	Learned Selector	Oracle Best
Directional	214.50	193.25
Uniform	287.58	286.08

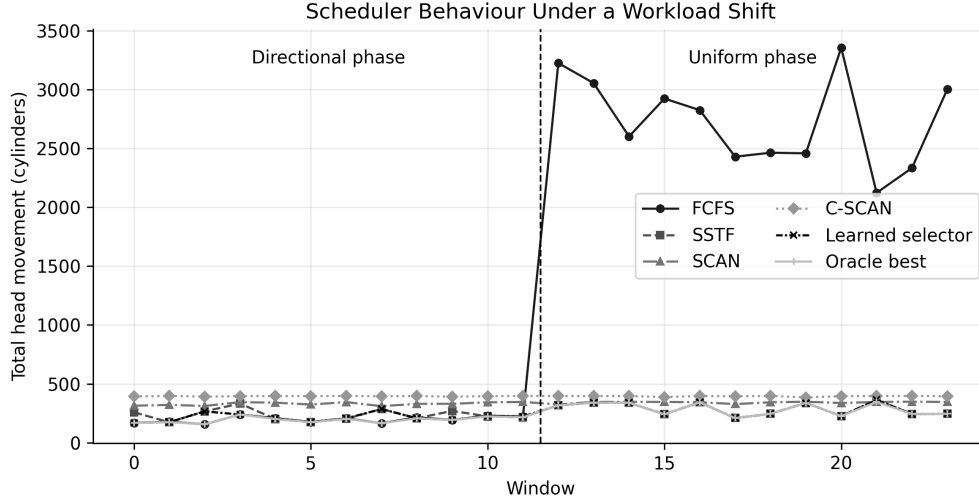


Figure 3: Behavior across the directional-to-uniform workload shift.

The selector made a fresh prediction for every window. During the uniform phase it remained close to the oracle, while the directional phase produced a larger difference.

4. Discussion

FCFS depends completely on arrival order. Under uniform workloads, this caused repeated long-distance movement. Directional workloads already contained useful spatial order, so FCFS became much more competitive.

SSTF performed strongly for uniform, localized, and bursty workloads. Localized requests provide many nearby choices, while bursty workloads contain short periods of locality. Even under uniform workloads, SSTF can reorganize an inefficient request order into shorter local movements. This explains why SSTF produced 535 of the 640 dataset labels.

SCAN and C-SCAN were penalized by the finite-window total-movement metric. The implementations use true endpoint behavior instead of LOOK or C-LOOK. SCAN therefore includes movement to the endpoint, while C-SCAN also includes wrap-around movement. This does not mean these algorithms are universally worse. A real operating system may also value fairness, waiting time, starvation avoidance, and request arrival timing.

The learned selector followed patterns present in its training data. It worked particularly well for workload types dominated by SSTF. Directional workloads

were harder because FCFS could outperform SSTF when the original request order already followed a useful path.

The class imbalance also limits the interpretation of the 84.38% accuracy. C-SCAN had no winning dataset labels under the selected workload distribution and metric. Different workload generators, disk sizes, arrival models, or evaluation objectives could produce a different distribution.

The simulation does not model rotational latency, transfer time, caching, controller behavior, file-system layout, or kernel overhead. Total head movement is therefore only a simplified seek-distance proxy. The learned selector should also not be assumed to generalize beyond the workload distribution used in this experiment without further testing.

5. Conclusion

The experiment demonstrates that request pattern can substantially change disk-scheduler performance. SSTF was strongest for most generated workloads, while FCFS was competitive for directional requests. The Decision Tree selector often selected an effective scheduler and stayed close to the oracle in several cases, but it was not always optimal. The results suggest that lightweight workload-aware selection can complement classical scheduling policies, although its effectiveness depends on the workload and prediction quality.

References

- [1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: John Wiley and Sons, 2018.
- [2] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books, 2018. [Online]. Available: <https://pages.cs.wisc.edu/~remzi/OSTEP/>
- [3] P. J. Denning, “The working set model for program behavior,” *Communications of the ACM*, vol. 11, no. 5, pp. 323–333, 1968, doi: 10.1145/363095.363141.
- [4] scikit-learn developers, “DecisionTreeClassifier,” *scikit-learn 1.7.2 documentation*. [Online]. Available: <https://scikit-learn.org/1.7/modules/generated/sklearn.tree.DecisionTreeClassifier.html>. Accessed: Oct. 3, 2026.